Sri Eshwar College of Engineering
An Autonomous Institution
Affiliated to Anna University, Chennai

Test Case Generator from User Story

Beyond Manual Test Design: Multi-LLM Agents for Automated Test Case Generation, Validation, and Automation

Presented By

Harshini T

Hema Dharshini M

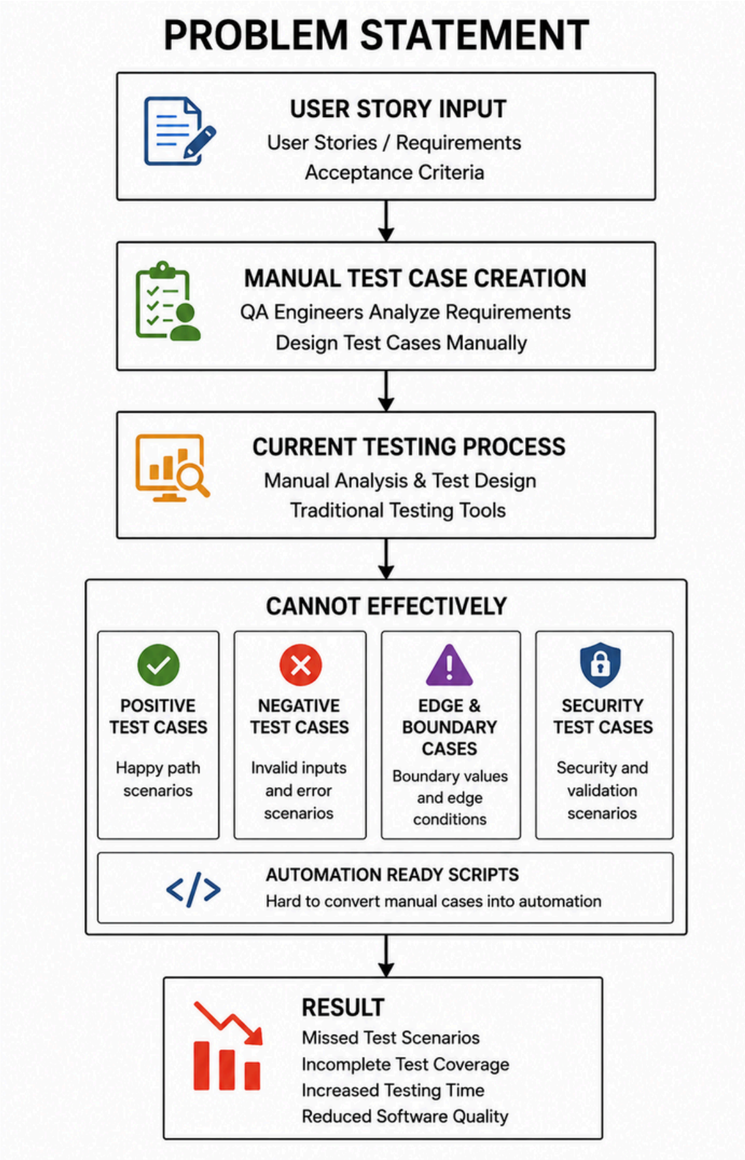
Hrithick Ram R

Jeyarani M

Problem Statement

Existing Challenge:
Need for an AI-driven testing platform that understands software requirements and automatically generates optimized test cases.
Modern software projects generate large numbers of user stories and feature requirements, making manual test case creation slow and error-prone.

Current Testing Systems Cannot Effectively:
- Analyze user stories automatically
- Generate complete test coverage
- Identify edge and negative scenarios
- Compare outputs from multiple AI models
- Create automation-ready test scripts
- Improve test quality through intelligent evaluation



Current Challenges

Requirement Analysis Challenges

User stories require manual analysis and interpretation.

Acceptance criteria may be incomplete or unclear.

Understanding complex business requirements takes time.

Important requirements can be overlooked.

Test Case Design Challenges

- Creating test cases manually is time-consuming.
- Positive, negative, and edge cases may be missed.
- Maintaining complete requirement coverage is difficult.
- Test quality depends on individual tester experience.

Testing Limitations

Current testing approaches often cannot:

- Automatically generate comprehensive test scenarios.
- Detect edge cases and boundary conditions effectively.
- Ensure full requirement traceability.
- Optimize test cases for automation readiness.

Operational Impact

- Increased testing effort and cost.
- Slower QA and release cycles.
- Higher risk of production defects.
- Reduced software quality and reliability.
- Lower productivity of QA teams.

Limitations of Existing Monitoring Systems

Key Limitations:
Current testing solutions lack:
AI-powered requirement understanding
Automated test case generation
Intelligent edge case discovery
Multi-LLM validation and comparison
Test quality optimization
Automation-ready script generation
Intelligent coverage analysi

Traditional Testing	TestForge AI
Manual test design	AI-generated test cases
Human analysis	AI requirement understanding
Limited coverage	Comprehensive coverage
Missed edge cases	Edge case detection
Manual automation	Automation-ready scripts
Single approach	Multi-LLM intelligence
Time-consuming	Fast and scalable

Proposed Solution

TestForge AI – AI Agentic Test Case Generation & Automation Platform

- Requirement Analyzer Agent

Analyzes user stories, acceptance criteria, and extracts functional requirements automatically.

- Multi-LLM Test Generation

Uses Gemini, Llama, and DeepSeek agents to generate independent test scenarios.

- AI Judge Agent

Evaluates generated test cases based on coverage, quality, edge cases, and automation readiness.

- Test Optimization Engine

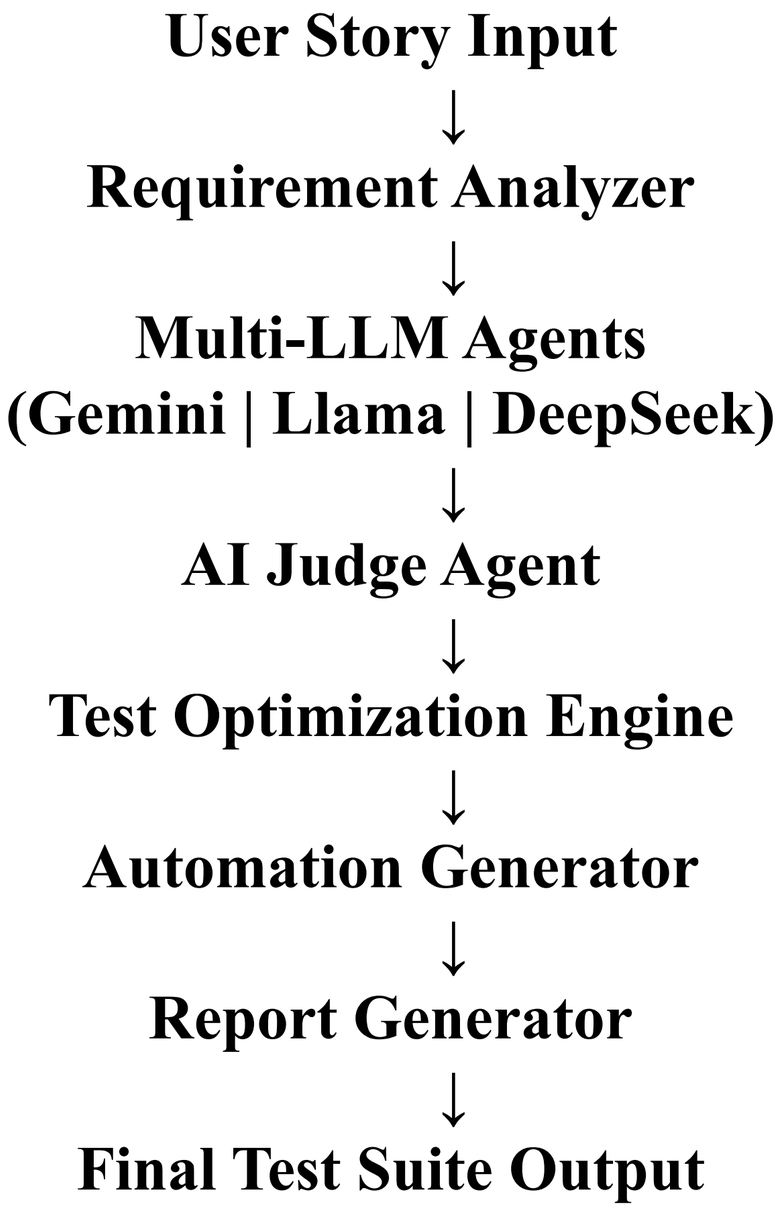
Removes duplicate scenarios and improves test coverage and effectiveness.

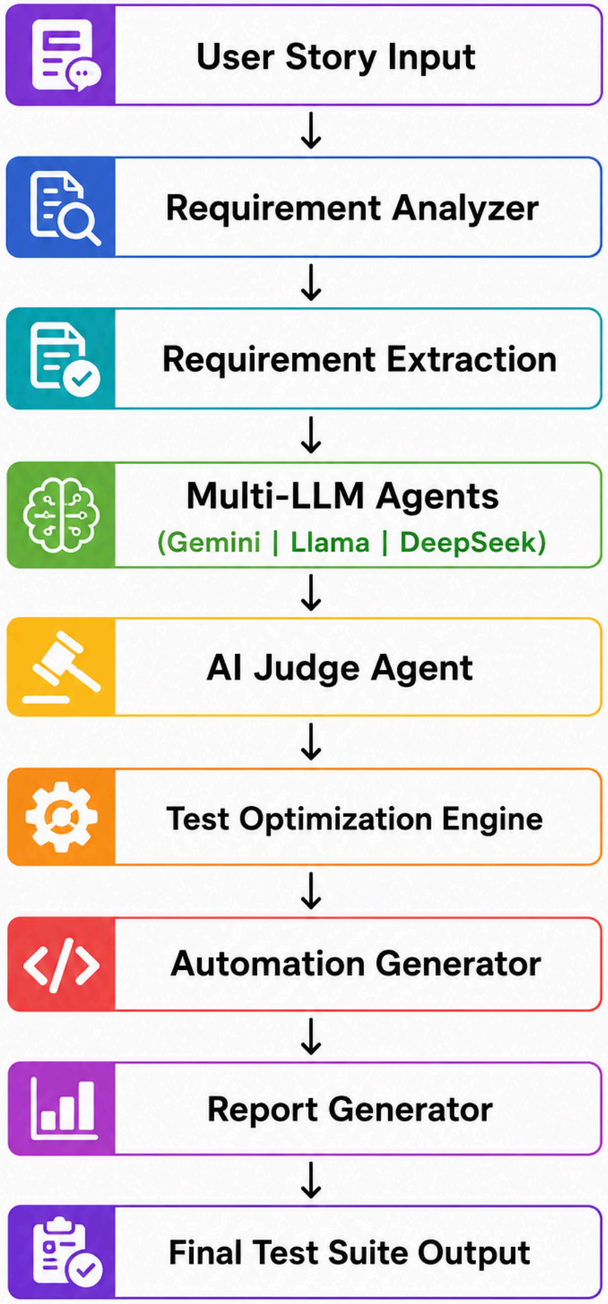
- Automation Script Generator

Generates Selenium, Playwright, Cucumber, and automation-ready test scripts.

- Report & Coverage Dashboard

Provides test coverage reports, quality scores, execution results, and downloadable artifacts.





System Architecture

Requirement Analyzer Agent
Analyzes user stories, extracts actors, requirements, acceptance criteria, and identifies missing information.

Multi-LLM AI Framework
Uses Gemini, Llama, and DeepSeek agents to generate independent test scenarios and testing strategies.

AI Judge & Optimization Engine
Evaluates generated outputs, selects the best scenarios, removes duplicates, and improves coverage.

Automation Generation Layer
Converts optimized test cases into Selenium, Playwright, Cucumber, and automation-ready scripts.

Reporting & Quality Insights
Generates coverage reports, quality scores, execution summaries, and downloadable artifacts.

"TestForge AI architecture enables intelligent requirement analysis, multi-LLM test generation, automated test creation, and quality-driven software testing workflows."

Key Features

1. User Story Analyzer

Analyzes user stories and extracts actors, requirements, and acceptance criteria automatically.

2. Multi-LLM Test Generation

Uses Gemini, Llama, and DeepSeek agents to generate multiple test strategies.

3. AI Judge Agent

Evaluates and ranks generated test cases based on quality and coverage.

4. Positive & Negative Test Case Generation

Automatically creates positive, negative, and validation test scenarios.

5. Edge & Boundary Case Detection

Identifies critical edge cases and boundary conditions often missed manually.

6. Test Optimization Engine

Removes duplicate test cases and improves overall test coverage.

7. Automation Script Generation

Generates Selenium, Playwright, and Cucumber-ready automation scripts.

8. Gherkin Feature File Generation

Creates BDD-compatible feature files automatically.

9. Test Coverage & Quality Reporting

Provides coverage analysis, quality scores, and execution insights.

10. Scalable AI Agent Architecture

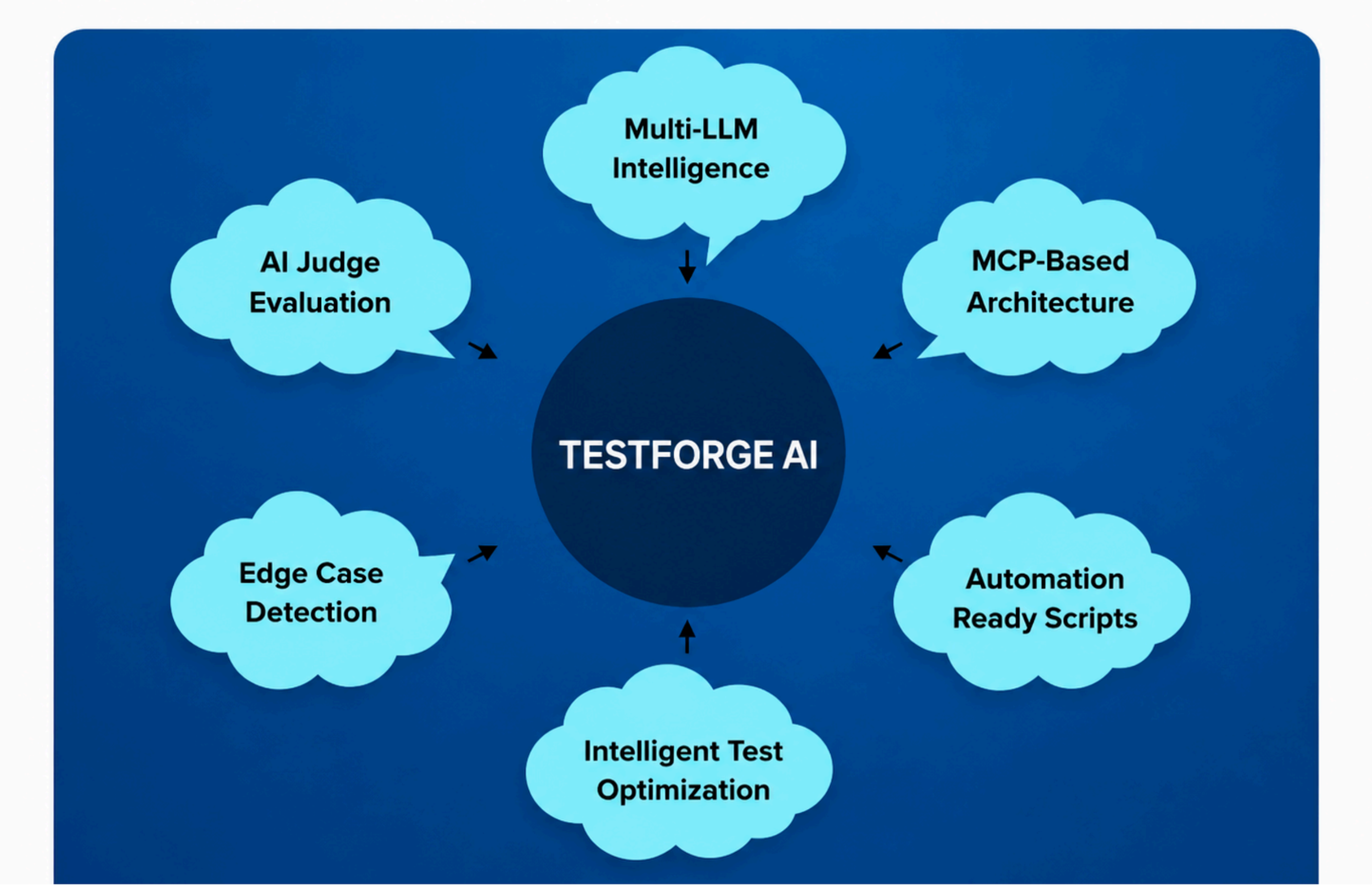
Supports modular AI workflows using MCP-based tool integration.

Technology Stack

AI Intelligence Workflow

Frontend Development	Backend Development	AI & Multi-LLM Engine
React.js	FastAPI	Gemini
Tailwind CSS	Python	Llama
HTML5	REST APIs	DeepSeek
JavaScript	JWT Auth	AI Judge
Database & Storage	Automation & Testing	Reporting & Optimization
MongoDB	Selenium	Coverage Analysis
Test History	Playwright	Quality Scoring
JSON Storage	Cucumber	Test Reports
Project Data	Gherkin	Export Generatio

Innovation


Multi-LLM Intelligence
AI Judge Evaluation
MCP-Based Architecture
TESTFORGE AI
Edge Case Detection
Automation Ready Scripts
Intelligent Test Optimization

Expected Impact

Phase 1 – Intelligent Requirement Analysis

- Automatically analyzes user stories and acceptance criteria.
- Identifies actors, requirements, and missing information.
- Reduces manual requirement understanding effort.
- Improves requirement traceability and clarity.

Phase 2 – Intelligent Anomaly Detection

- Generates positive, negative, edge, and boundary test cases.
- Ensures broader and more consistent test coverage.
- Reduces manual test design time significantly.
- Detects scenarios often missed during manual testing.

Phase 3 – AI-Powered Test Optimization

- Evaluates test quality using AI Judge Agent.
- Removes duplicate and redundant test cases.
- Improves automation readiness of test suites.
- Enhances overall testing effectiveness.

Phase 4 – Faster Software Delivery

- Accelerates QA and release cycles.
- Improves defect detection and software reliability.
- Reduces testing effort and operational cost.
- Supports high-quality and faster software releases.